

DAY 6 — Lead/VP Engineering: Architecture, Strategy, Governance and Technical Leadership

At Lead/Staff/Principal/VP level, engineering stops being mainly about **producing correct code** and becomes increasingly about creating an environment in which **many teams can repeatedly produce correct, secure, reliable and economically sensible systems**.

A useful progression is:

Junior/Mid Engineer

↓

"Can I implement this correctly?"

Senior Engineer

↓

"Can I design this service correctly?"

Staff / Principal

↓

"Can several services and teams evolve coherently?"

Engineering Lead / Director

↓

"Can the organisation deliver and operate this capability sustainably?"

VP Engineering

↓

"Are we investing engineering capacity in the right capabilities, with the right architecture, risk posture and operating model?"

The biggest shift is therefore:

| Implementation thinking → system thinking → organisational thinking → enterprise thinking.

1. Engineering scope progression

Consider these scopes:

Code

↓

Component

↓

Service

↓

System

↓

Platform

↓

Organisation

↓

Enterprise

Code

Concern:

- correctness
- readability
- algorithms
- tests
- error handling

Example:

Python function:
`generate_router_configuration(device)`

You mainly ask:

| Does this function behave correctly?

Component

A component performs a bounded responsibility.

Example:

Configuration Generator

Now you care about:

- interfaces
 - internal dependencies
 - extensibility
 - testing
 - maintainability
-

Service

Example:

Configuration Generation Service

Now additional concerns appear:

- APIs
- deployment
- scaling
- availability
- authentication
- databases
- observability
- operational ownership

Your question becomes:

| Can another system safely depend on this service?

System

A network automation system may contain:

API
Inventory
Workflow Engine
Queue
Workers
Policy Engine
Device Adapters
Audit Store
Telemetry

Now architecture matters considerably more.

A perfectly implemented worker is irrelevant if the overall workflow can accidentally configure 10,000 routers simultaneously.

Platform

A platform supports **many use cases and many teams**.

For example:

Network Automation Platform

- VLAN provisioning
- Firewall automation
- Router upgrades
- Configuration compliance
- Incident remediation
- Capacity changes

Now you must think about:

- self-service
 - reusable capabilities
 - tenancy
 - guardrails
 - standards
 - platform APIs
 - user experience
 - governance
 - adoption
-

Organisation

At organisational scope, architecture and team structure interact.

Questions become:

- Who owns the platform?
 - Who owns production incidents?
 - Who decides standards?
 - Which teams depend on which teams?
 - Where are bottlenecks?
 - Who can approve high-risk automation?
-

Enterprise

Enterprise scope introduces:

- business continuity
- regulatory requirements
- third-party risk
- financial planning
- security policy
- technology strategy
- enterprise-wide standards
- multi-year migration
- portfolio prioritisation

At this level, a technically excellent architecture can still be wrong if it creates unacceptable operational or financial risk.

2. Lead/Staff/VP technical thinking

An implementation-oriented engineer often begins with:

| How should I implement this?

A Lead/Staff/VP engineer should first ask:

Why?

↓

Who needs it?

↓

What problem exists?

↓

What constraints exist?

↓

How important is the problem?

↓

What risks exist?

↓

What solution options exist?

↓

Which trade-offs are acceptable?

↓

Who builds it?

↓

Who operates it?

↓

How much does it cost?

↓

How does it evolve?

Suppose someone requests:

| "Build a microservice for router configuration."

Implementation thinking immediately considers FastAPI, Kubernetes and PostgreSQL.

Leadership thinking asks first:

| Why do we need a separate service?

Perhaps the actual problem is simply that three teams are duplicating configuration-generation logic.

The correct solution might therefore be:

- shared library
- existing platform extension
- internal API
- workflow capability

rather than another microservice.

Technology selection comes **after problem understanding**.

3. Problem framing

A good technical problem statement separates several dimensions.

Business problem

Why does the organisation care?

Example:

Network changes currently require two days of manual work, preventing rapid service activation.

User/operator problem

What difficulty does the user experience?

Example:

Network engineers manually validate device state, generate commands and coordinate execution.

Technical problem

What system limitation causes the problem?

Example:

There is no authoritative inventory, standard automation API or durable workflow engine.

Constraints

Examples:

- regulated environment
 - legacy routers
 - limited maintenance windows
 - private-cloud requirement
 - existing HSM infrastructure
 - 99.95% availability requirement
 - budget limits
-

Dependencies

Examples:

Network Automation

```
|
+--> CMDB
+--> IPAM
+--> IAM
+--> Network Controllers
+--> DNS
+--> Secrets platform
```

Dependencies may dominate your architecture.

Assumptions

Example:

| Devices support transactional configuration.

That is an assumption.

If many devices do not, rollback architecture changes substantially.

Unknowns

Examples:

- actual peak job volume
- percentage of legacy devices
- controller API limits
- future regional growth

Good engineering makes unknowns visible rather than silently converting them into assumptions.

Why weak framing is dangerous

Imagine building an advanced automation engine capable of processing one million jobs per hour.

The real organisation may run only:

20,000 network changes/month

while its biggest problem is actually **poor inventory accuracy**.

You would have produced excellent technology that solves the wrong problem.

4. Functional and non-functional requirements

Functional requirements

Functional requirements describe **what the system must do**.

For a network automation platform:

- register devices
- discover state
- generate configuration
- approve changes
- execute changes
- validate outcomes
- rollback
- expose APIs

- maintain audit history
-

Non-functional requirements

NFRs describe **how well or under what constraints** the system must operate.

Availability

How often must the service be usable?

Example:

99.95% monthly availability

Scalability

Can it support growth?

Example:

10,000 devices today
100,000 devices in 3 years

Performance

Example:

API p95 latency < 300 ms

Not every operation needs low latency.

A firmware rollout might legitimately take hours.

Reliability

Does the system perform the requested operation correctly?

For automation this can be more important than API latency.

Security

Examples:

- strong identity
 - least privilege
 - secret protection
 - encryption
 - device credential management
-

Maintainability

Can teams change the system safely?

Operability

Can production teams understand and operate it?

Observability

Can failures be detected and diagnosed?

Auditability

Can you prove:

who
changed what
on which device
when
why
using which approval
with what result

Cost

What is the acceptable total cost?

Recoverability

How quickly can service and data be restored?

NFRs drive architecture

Consider two automation platforms.

Platform A

500 lab devices
best-effort availability
no regulatory requirement

A single-region architecture may be perfectly reasonable.

Platform B

100,000 production devices
financial institution
global operation
strict audit requirements

It probably requires:

- regional failure isolation
- durable workflows
- strong audit trails
- DR
- strict IAM
- controlled deployment
- approval policies

Same functionality.

Very different architecture.

5. Technical strategy

Technical strategy answers:

| How should technology capabilities evolve so the organisation can achieve its goals?

A useful structure is:

Current State

↓

Desired State

↓

Guiding Principles

↓

Capability Gaps

↓

Constraints

↓

Architectural Direction

↓

Migration Approach

↓

Investment Priorities

Current state

Example:

Scripts owned by individual teams
Manual approvals
Local credentials
Inconsistent logging
No central inventory

Desired state

Central automation platform
API-driven
Policy controlled
Auditable
Self-service
Resilient

Guiding principles

Examples:

- API first
 - automation by default
 - least privilege
 - source of truth driven
 - incremental migration
 - explicit ownership
 - managed services where appropriate
-

Capability gaps

Perhaps you lack:

- inventory
 - workflow engine
 - observability
 - automated testing
 - secrets management
-

Architectural direction

For example:

Move from team-specific scripts toward an API-driven, event-capable automation platform using durable workflows and controlled worker pools.

Notice that this describes direction without unnecessarily prescribing every product.

Investment priorities

Perhaps:

1. Inventory quality
 2. Standard API
 3. Workflow reliability
 4. Security controls
 5. Self-service experience
-

Strategy is not a technology shopping list

This is not technical strategy:

| Kubernetes + Kafka + PostgreSQL + Redis + Go.

Those are technologies.

Strategy explains:

| Why are we changing, toward what state, according to which principles, and through what migration path?

6. Strategy vs architecture vs roadmap vs project plan

These are frequently confused.

Concept	Main question
Strategy	Where and why are we going?
Architecture	How should the system be structured?
Roadmap	In what sequence will capabilities appear?
Project plan	Who does which work and when?

Example:

Strategy

| Standardise network changes through one governed automation platform.

Architecture

API → Workflow → Queue → Workers → Devices

Roadmap

Q1 Inventory
Q2 Configuration automation
Q3 Compliance
Q4 Closed-loop remediation

Project plan

Team A → Inventory API
Team B → workflow engine
Team C → controller adapters

Relationship:

Business Goals



Technical Strategy



Architecture



Roadmap



Projects

7. Architecture decision-making

Architecture decisions should be based on explicit criteria rather than technology preference.

Evaluate:

- simplicity
- reliability
- scalability
- security
- performance
- operability
- team capability
- cost
- migration complexity
- vendor dependency
- reversibility

Suppose choosing a workflow engine.

Criterion	Option A	Option B
Simplicity	High	Medium
Reliability	Medium	High
Scale	Medium	High
Operational burden	Low	Medium
Team experience	High	Low
Vendor lock-in	Low	Medium
Migration cost	Low	High

The point is not necessarily mathematical scoring.

The important discipline is:

| Make the criteria and trade-offs explicit.

8. Architectural trade-offs

Architecture consists largely of trade-offs.

Synchronous vs asynchronous

Sync

Client → Service → Device → Response

Simple.

Useful when execution is:

- short
- predictable
- reliable

But network changes may take minutes.

Async is often better:

```
Client
  |
submit
  v
Job API → Queue → Worker
  |
job ID
  v
Client
```

Trade-off:

Async improves resilience and scalability
but increases state-management complexity.

SQL vs NoSQL

SQL provides:

- transactions
- relational integrity
- rich queries

NoSQL may provide:

- flexible schema
- scale characteristics
- domain-specific access patterns

Neither is universally superior.

Monolith vs microservices

A monolith provides:

- simple deployment
- easy transactions
- lower operational complexity

Microservices provide:

- independent ownership
- isolated deployment
- scaling flexibility

But add:

- networking
- distributed tracing
- API compatibility
- retries
- deployment coordination
- data consistency issues

A well-designed modular monolith is often better than prematurely creating dozens of services.

Build vs buy

Building gives control.

Buying may accelerate delivery.

But either can be expensive.

Managed vs self-managed

Managed Kafka, for example, reduces operational effort but creates:

- vendor cost
- integration constraints
- some lock-in

Self-managed Kafka gives control but now your team owns Kafka failures.

Strong vs eventual consistency

Strong consistency simplifies reasoning but may reduce availability/performance in some distributed scenarios.

Eventual consistency provides flexibility but means:

system A says version 11
system B temporarily says version 10

Applications must tolerate this.

Centralised vs distributed control

Centralised control improves consistency.

Distributed control improves autonomy and failure isolation.

Fast change vs controlled change

Fast change improves delivery velocity.

Controls reduce production risk.

Strong organisations design pipelines so that **safety is automated**, allowing both reasonable speed and control.

9. Architecture principles

Principles guide repeated decisions.

High cohesion

Things belonging together should remain together.

Example:

Device configuration logic + validation rules

may belong in the same domain.

Loose coupling

Systems should know as little as necessary about one another.

Encapsulation

Hide implementation details behind stable interfaces.

Clear ownership

Every critical capability needs an accountable owner.

Stable interfaces

Consumers should depend on contracts rather than internal implementation.

Separation of concerns

Examples:

API handling
workflow orchestration
device execution
audit storage

should not become one giant module.

Failure isolation

Failure of one device or customer should not collapse the entire platform.

Idempotency

Repeating an operation should not create unintended duplicate effects.

Especially critical for:

POST /network-changes

when clients retry.

Automation by default

Repeated operational work should progressively become automated.

Security by design

Security should exist in architecture, not as a final review.

Observability by design

Design events, metrics, traces and audit records along with functionality.

10. Domain and service boundaries

A good service owns a meaningful business or technical responsibility.

Consider:

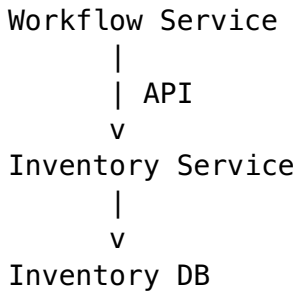
Inventory Service

It should own:

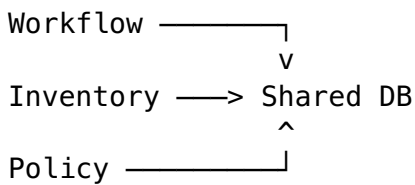
- device identity
- device metadata
- lifecycle information

A workflow service should not directly manipulate Inventory's database.

Better:



Avoid:



A shared database creates hidden coupling.

A schema change by one team may break another.

Good boundaries clarify:

- Capability ownership
- API ownership
- Data ownership
- Dependency direction

11. Evolutionary architecture

Long-lived systems will change.

Therefore architecture should support evolution.

Incremental evolution

Build capabilities gradually.

Backward compatibility

New APIs should avoid unnecessarily breaking old consumers.

Example:

```
v1:
{
  "device_id": "R1"
}
```

```
Compatible addition:
{
  "device_id": "R1",
  "region": "APAC"
}
```

Versioning

Sometimes incompatible changes require:

```
/v1/jobs
/v2/jobs
```

Versioning carries cost because multiple versions may need support.

Strangler pattern

Instead of replacing a legacy system instantly:

```
Client → API —┐ Legacy capability
                  └─ New capability
```

Gradually move responsibilities.

```
10% new
↓
30%
↓
70%
↓
100%
```

Then retire the legacy system.

Migration

A migration needs consideration of:

- data
- APIs
- traffic
- users
- operational procedures

- observability
 - rollback
-

Deprecation

Deprecation should be deliberate:

announce
↓
migration guidance
↓
usage monitoring
↓
deadline
↓
shutdown

Avoid big-bang rewrites

Large rewrites often underestimate:

- hidden behaviour
- edge cases
- operational knowledge
- integration complexity

Evolutionary replacement generally reduces risk.

12. Architecture runway

Architecture runway means:

Build enough foundational capability so upcoming product delivery is not repeatedly blocked.

Suppose future teams need network automation.

Before all features exist, you may establish:

- common API gateway
- authentication
- audit event model
- basic workflow framework
- CI/CD
- logging standards

That creates runway.

But creating a ten-year generic platform before having real users is over-engineering.

Balance:

Too little foundation



delivery repeatedly blocked

Enough runway



future development enabled

Too much speculative foundation



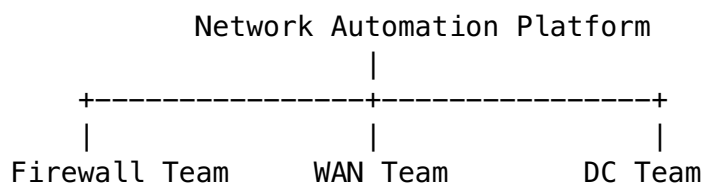
cost + complexity + unused abstractions

13. Platform engineering

An application generally solves a specific user problem.

A platform enables **many teams to solve their own problems** using common capabilities.

Example:



The platform provides:

- APIs
- execution engine
- authentication
- policy enforcement
- audit
- secrets
- telemetry
- workflow capabilities

Teams build use cases on top.

Internal customers

Your developers, network engineers and operators are customers.

Self-service

Instead of submitting tickets:

Engineer → Portal/API → Automation

Golden paths

A recommended way to perform common tasks.

For example:

Use the standard network-change workflow that already includes approval, validation, audit and rollback.

Guardrails

Guardrails constrain dangerous actions without forcing every decision through a central team.

Example:

Low-risk change

↓

automatic approval

High-risk core-router change

↓

manual approval

Product mindset

A platform should have:

- users
- roadmap
- adoption goals
- usability
- documentation
- feedback loops
- support model

Building shared infrastructure and declaring it "a platform" does not create one.

14. Build vs buy

A useful decision framework:

Strategic differentiation

Is this capability something that differentiates your organisation?

If not, buying may be sensible.

Time to market

Can a vendor deliver capability much sooner?

Total cost

Compare:

Licence
+ integration
+ infrastructure
+ operations
+ engineering
+ support
+ migration
+ exit

not simply licence cost.

Vendor maturity

Ask:

- proven at required scale?
 - enterprise support?
 - security maturity?
 - roadmap stability?
-

Integration

A great product that integrates badly may be expensive.

Compliance

Can the vendor meet regulatory requirements?

Operational ownership

Even SaaS creates internal responsibilities.

Someone still owns:

- configuration
 - access
 - integration
 - incident escalation
-

Lock-in

How difficult would migration be?

Exit strategy

Ask before purchase:

| If this vendor becomes unacceptable in five years, how do we leave?

15. Technical debt

Technical debt is a future cost created by today's engineering decision.

Intentional debt

Example:

| For launch, we use one regional queue because current scale is small. We know multi-region support will be required later.

This can be rational.

Accidental debt

Created through:

- weak design
 - inadequate testing
 - inconsistent standards
 - undocumented dependencies
-

Debt interest

Debt creates ongoing cost:

slower feature delivery
more defects
higher operational effort
riskier releases
longer onboarding

Rational debt

Suppose:

Perfect architecture = 8 months
Adequate architecture = 3 months

and the business must validate market demand.

Taking architectural debt may be sensible.

But it should be:

Known
Visible
Owned
Measured
Planned

Technical debt becomes dangerous when the organisation pretends it does not exist.

16. Architecture governance

Governance should help the organisation make good technical decisions repeatedly.

Healthy governance uses:

- principles
- standards
- reviews
- guardrails
- documented exceptions
- ownership
- decision history

Bad governance becomes:

Meeting

↓

another meeting

↓

committee

↓

PowerPoint

↓

approval

Healthy governance asks:

| Is this decision high risk enough to require additional review?

Example:

A new logging library might require no architecture board.

A new enterprise identity system probably does.

Governance should be **proportional to risk**.

17. Architecture Decision Records

ADR = Architecture Decision Record.

A simple ADR contains:

Context

Why is a decision required?

Decision

What did we decide?

Alternatives

What else was considered?

Consequences

What do we gain and what cost do we accept?

Example:

ADR-027

Use durable queue for network-change execution

Context:

Device changes may take minutes and cannot depend on HTTP connection lifetime.

Decision:

API creates durable jobs consumed by worker pools.

Alternatives:

1. Synchronous HTTP execution
2. In-memory task execution

Consequences:

- + reliable retries
- + horizontal worker scaling
- + failure recovery
- queue infrastructure required
- more complex job-state model

Years later, engineers can understand **why** the system looks the way it does.

That is often more valuable than simply knowing what exists.

18. Standards and golden paths

Platforms usually standardise:

- APIs
- logging
- authentication
- deployment
- observability
- testing
- reliability

Example API standard:

Authentication
Correlation ID
Error format
Pagination
Versioning
Idempotency

Teams should not reinvent these.

A golden path might provide:

service template
+ CI pipeline
+ security scan
+ logging
+ metrics
+ deployment manifest
+ dashboards

New teams gain production readiness quickly.

But excessive standards can block experimentation.

A useful model is:

Default path = strongly supported
Alternative path = allowed with justification
Dangerous path = restricted

19. Risk-based engineering

Simple conceptual model:

$\text{Risk} \approx \text{Likelihood} \times \text{Impact}$

Suppose automation could incorrectly update 20,000 routers.

Even if probability is low, impact may be enormous.

Risk management asks six questions:

Identify
↓
Reduce likelihood
↓
Reduce impact
↓
Limit blast radius
↓
Detect quickly
↓
Recover quickly

Example controls:

Reduce likelihood

- validation
- testing
- policy checks

Reduce impact

- staged rollout

Limit blast radius

10 routers

→ 100

→ 1,000

→ remainder

Detect

- telemetry
- health checks
- anomaly detection

Recover

- rollback
- configuration restoration
- manual override

Strong engineering rarely assumes:

| Failure will never happen.

It asks:

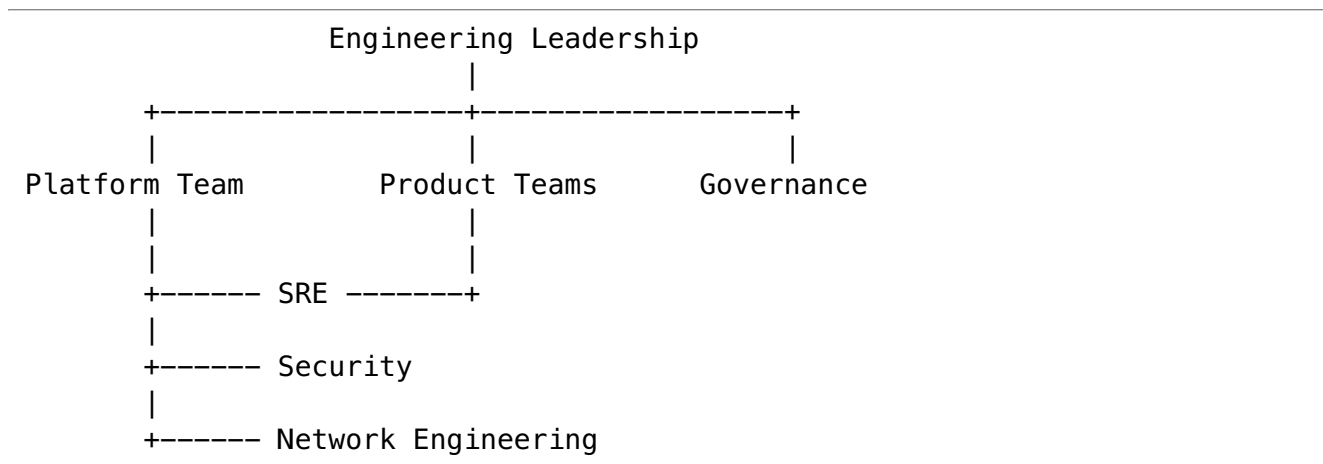
| When failure happens, how bad can it become?

20. Engineering operating model

Architecture defines system structure.

The operating model defines **how people organise around that system.**

Possible structure:



Platform team

Owns:

- shared capabilities
- platform APIs
- developer experience
- platform reliability

Product/application teams

Build business workflows using the platform.

SRE/operations

Focus on:

- production reliability
- observability
- incident response
- capacity

Security

Defines and validates security controls.

Network engineering

Provides domain expertise:

- device behaviour
- topology
- vendor specifics
- operational requirements

Governance/risk

Ensures appropriate evidence, control and accountability.

Critical principle:

| Collaboration does not mean unclear ownership.

For every important capability, someone should be accountable.

21. RACI

RACI is a lightweight responsibility model.

- **R** — Responsible: performs the work.
- **A** — Accountable: ultimately owns the outcome.
- **C** — Consulted: provides input.
- **I** — Informed: needs awareness.

Example:

Activity	Platform	Network	Security	SRE
Workflow engine	A/R	C	C	C
Device policy	C	A/R	C	I
Security standard	C	C	A/R	C
Production monitoring	C	C	I	A/R

Avoid turning RACI into administrative overhead.

Its purpose is simply to remove ambiguity.

22. Cross-functional platform delivery

A network automation platform crosses many disciplines.

Software Engineering

|
Network Engineering

|
Cloud / Kubernetes

|
SRE

|
Security

|
Data

|
Operations

|
Risk

Problems occur when interfaces are unclear.

Example:

Who owns device certificates?

Could be:

Network team?

Security?

Platform?

PKI team?

If nobody knows, incidents become slow.

Strong leadership establishes:

- interface contracts
 - ownership boundaries
 - escalation paths
 - dependency visibility
-

23. Stakeholder thinking

Different stakeholders optimise for different things.

Engineers

Care about:

- maintainability
- technical quality
- developer velocity

Architects

Care about:

- system coherence
- long-term structure
- standards

Operations

Care about:

- stability
- troubleshooting
- predictable change

Security

Care about:

- attack surface
- access
- data protection

Risk

Care about:

- control
- evidence
- accountability

Product

Care about:

- capabilities
- customer outcomes
- delivery speed

Finance

Care about:

- cost
- predictability
- ROI

Business leadership

Care about:

- growth
- risk
- strategic outcomes

A good technical decision usually acknowledges several simultaneously.

24. Influencing without direct authority

Senior technical leaders rarely control everyone whose cooperation they need.

Influence comes from several sources.

Credibility

Consistently demonstrate technical judgement.

Evidence

Replace:

| "Microservices are better."

with:

| "Our current deployment requires six teams to coordinate one release. These three boundaries would permit independent deployment."

Shared outcomes

Instead of:

| "Security is blocking engineering."

Frame:

| "How can we achieve the required control while reducing approval latency?"

Understand incentives

Operations may resist automation because they fear uncontrolled changes.

That is rational if automation has weak rollback.

Solve the underlying concern.

Technical negotiation

Architecture discussions should compare consequences rather than personalities.

Decision transparency

After disagreement:

Context
Options
Decision
Reason
Trade-offs

Everyone may not prefer the outcome, but they can understand it.

25. Communicating technical complexity

Suppose:

| Network automation workers occasionally retry timed-out operations, creating a possibility that the same configuration command is executed twice.

Different audiences need different communication.

Engineer

The device operation isn't idempotent. A timeout after device execution but before acknowledgement can cause duplicate execution. We need operation IDs or reconciliation against device state.

Architect

Our current execution contract gives at-least-once delivery, but some device actions aren't safely repeatable. We need an idempotency and reconciliation strategy across adapters.

Operations

A timeout can occasionally create a duplicate configuration attempt. We're adding protection so retries first verify actual device state.

Security/risk

Certain failure conditions could result in an unintended repeated network change. The control enhancement adds execution identifiers, state verification and audit evidence before retry.

Business leader

We found a reliability scenario where an automated network change could be repeated during a timeout. We're reducing that operational risk before increasing automation scale.

Same issue.

Different abstraction.

26. Engineering quality as a system

Quality should not rely on brilliant engineers manually catching everything.

Build a quality system.

Design

↓

Architecture review

↓

Code

↓

Code review

↓

Automated tests

↓

Security checks

↓

CI/CD

↓

Controlled deployment

↓

Observability

↓

Incident learning

↓

Design improvement

Quality mechanisms include:

- automated testing
- code review
- architecture review
- documentation
- standards
- CI/CD
- observability
- ownership
- incident learning

Hero culture sounds like:

| Ravi always fixes production.

Healthy engineering asks:

| Why does the system require Ravi to rescue production?

The objective is **repeatable organisational capability**.

27. Growing engineers

A senior leader multiplies engineering capability.

Delegation

Delegate meaningful outcomes, not just tasks.

Weak:

| Implement endpoint X.

Better:

| Own the configuration-validation capability, including design and operational readiness.

Technical coaching

Guide thinking rather than always providing answers.

Design reviews

Use reviews to teach:

- trade-offs
 - assumptions
 - failure thinking
 - operational consequences
-

Giving ownership

People grow by owning consequences.

Constructive feedback

Feedback should be:

- specific
 - timely
 - actionable
 - technically grounded
-

Knowledge sharing

Avoid critical knowledge remaining inside one person's head.

Developing specialists

Not everybody must become a manager.

Strong organisations need deep specialists in:

- networking
- security
- databases
- distributed systems
- SRE
- cloud

28. Long-term platform ownership

A platform lifecycle is much larger than development.

Design
↓
Build
↓
Deploy
↓
Operate
↓
Scale
↓
Secure
↓
Upgrade
↓
Modernise
↓
Migrate
↓
Deprecate
↓
Retire

Leadership responsibility includes:

Build

Correct architecture and implementation.

Deployment

Repeatable and safe delivery.

Operations

Support model and incident response.

Reliability

SLOs and resilience.

Security

Continuous vulnerability and access management.

Capacity

Growth planning.

Cost

Ongoing optimisation.

Upgrades

Runtime, OS, dependencies, Kubernetes, databases, vendors.

Migration

Move consumers safely.

Deprecation

Stop supporting obsolete capabilities.

End-of-life

Delete systems that no longer provide value.

Deleting old systems is often as important as building new ones.

29. Roadmapping and prioritisation

Engineering capacity is finite.

You may have:

Business features	40 requests
Security work	15 requests
Reliability improvements	12
Technical debt	25
Platform improvements	20
Migration work	10

Everything cannot be priority one.

Leadership balances:

Value
Risk
Urgency
Dependency
Effort
Strategic alignment

A healthy roadmap normally reserves meaningful capacity for:

- delivery
- reliability
- security
- debt
- strategic platform work

If 100% of capacity is allocated to business features indefinitely, eventually reliability and delivery velocity deteriorate.

30. Engineering economics

Technical architecture has economic consequences.

Consider:

Engineering cost
Infrastructure cost
Operational cost
Complexity cost
Failure cost
Migration cost
Vendor cost

Together they contribute to **total cost of ownership**.

Engineering effort

People time required to build and maintain something.

Infrastructure cost

Compute, storage, networking and licences.

Operational cost

On-call effort, support, upgrades and incident management.

Cost of complexity

Suppose introducing Kafka solves a scaling problem.

You now need expertise around:

- partitioning
- lag
- brokers
- schema compatibility
- monitoring
- upgrades

Complexity itself has a cost.

Cost of failure

A one-hour network outage could cost far more than several engineer-years.

That changes rational architecture investment.

CapEx

Capital expenditure traditionally involves long-lived assets.

Example:

Purchasing data-centre hardware.

OpEx

Operating expenditure is ongoing operating cost.

Example:

Monthly cloud service consumption.

FinOps

FinOps means creating visibility and accountability around cloud expenditure.

Useful questions include:

What does each workload cost?
Why did cost grow?
Who owns the cost?
Can capacity be optimised?

31. Capacity planning

Architectures should use explicit growth assumptions.

Suppose today:

Devices	20,000
API requests	100 req/s peak
Automation jobs	20,000/day
Telemetry	2 TB/day
Workers	50

Forecast:

Year 1	Year 3
20K devices	→ 80K
20K jobs	→ 200K jobs/day
2 TB	→ 10 TB telemetry/day

Now ask:

- Can the inventory database scale?
- Does telemetry retention remain affordable?
- How many workers are needed?

- What happens during peak maintenance windows?
 - What is controller API capacity?
-

Worker example

Suppose one worker processes:

12 jobs/minute

And peak demand is:

6,000 jobs/hour
= 100 jobs/minute

Minimum theoretical workers:

$100 / 12 \approx 9$

But architecture should also account for:

- retries
- slow devices
- failover
- capacity headroom

You might therefore provision significantly more.

Capacity planning is not predicting the future perfectly.

It means **making the assumptions visible**.

32. Vendor and third-party engineering

Third parties are part of your architecture.

Assess:

Dependency

How critical is the vendor?

Supportability

What support exists during incidents?

SLA

What does the provider actually guarantee?

Integration

How tightly is the system coupled?

Security

How is access controlled?

Data ownership

Who owns and can retrieve your data?

Lock-in

Can workloads move elsewhere?

Exit strategy

What happens if:

- pricing changes
 - vendor disappears
 - regulator objects
 - product is discontinued
-

Third-party risk

Especially in regulated enterprises, vendors may require:

- security assessment
 - resilience assessment
 - data-location assessment
 - continuity planning
 - contractual controls
-

33. Conway's Law

Conway's Law roughly says:

| Systems tend to reflect the communication structure of the organisations that build them.

Consider:

Network Team
Platform Team
Security Team
Operations Team

If teams rarely collaborate, architecture may look like:

Network System → Platform System → Security System → Ops System

with painful handoffs.

Team boundaries often influence:

Ownership

↓

Service boundaries

↓

APIs

↓

Data boundaries

↓

Operational responsibilities

This means architecture changes sometimes require organisational changes.

You cannot always fix a communication problem with an API.

34. Regulated-enterprise architecture

Regulated environments add significant requirements.

Auditability

You need evidence.

Security

Controls must be demonstrable, not merely assumed.

Operational resilience

Critical services must withstand disruptions.

Data protection

Understand:

What data?

Where stored?

Who accesses it?

How long retained?

Change governance

Production changes may require formal controls.

Business continuity

The organisation needs alternatives when systems fail.

Third-party risk

Critical vendors become part of enterprise resilience.

Evidence

A control without evidence may be insufficient.

For example:

Control:

Production changes require approval.

Evidence:

Approval ID linked to every production change.

Accountability

Named roles should own critical controls.

35. Measuring platform success

A platform should measure outcomes.

Adoption

How much of the target population actually uses it?

Automation platform supports 20 use cases

means little if nobody adopts them.

Reliability

Examples:

SLO attainment

change success rate

job success rate

Lead time

How long from requested capability to deployment?

Automation coverage

Example:

80% of standard network changes automated

Failure rate

What percentage of automation changes fail?

Recovery time

How quickly are failures restored?

Developer/operator productivity

Can users accomplish work faster?

Cost efficiency

Examples:

cost per managed device
cost per automation job
cost per telemetry GB

Avoid vanity metrics

Vanity:

| We processed 50 million API calls.

Outcome:

| Network provisioning time fell from two days to 12 minutes with a 99.8% successful-change rate.

The second tells you whether the platform creates value.

36. Integrated case study

Global Network Automation Platform

Assume a large regulated enterprise operates:

100,000+ network devices
60+ countries
multiple data centres
public cloud
branches
WAN
campus
firewalls
routers
switches

Current processes rely heavily on scripts and manual change execution.

We need a long-term architecture and engineering operating model.

Step 1 — Business need

Current problems:

- network provisioning takes days
- large manual effort
- inconsistent processes
- configuration drift
- audit evidence difficult to assemble
- network changes create operational risk
- different regions maintain separate scripts

Desired business outcomes:

Faster delivery
Lower operational effort
Safer changes
Consistent controls
Higher resilience
Better auditability

Step 2 — Users

Primary users:

- network engineers
- network operations
- application teams
- cloud teams
- SRE
- security teams

Secondary stakeholders:

- risk
 - audit
 - finance
 - engineering leadership
-

Step 3 — Functional requirements

Platform should support:

- device inventory
- configuration generation
- policy validation

- workflow orchestration
 - approval
 - scheduled execution
 - rollback
 - compliance checking
 - telemetry
 - audit
 - APIs
 - self-service
-

Step 4 — NFRs

Example requirements:

Control-plane availability	99.95%+
Job durability	extremely high
Audit retention	multi-year
Regional blast-radius isolation	required
Security	enterprise IAM + least privilege
Recovery	defined RT0/RP0
Scale	>100K devices

More importantly:

| Automation must fail safely.

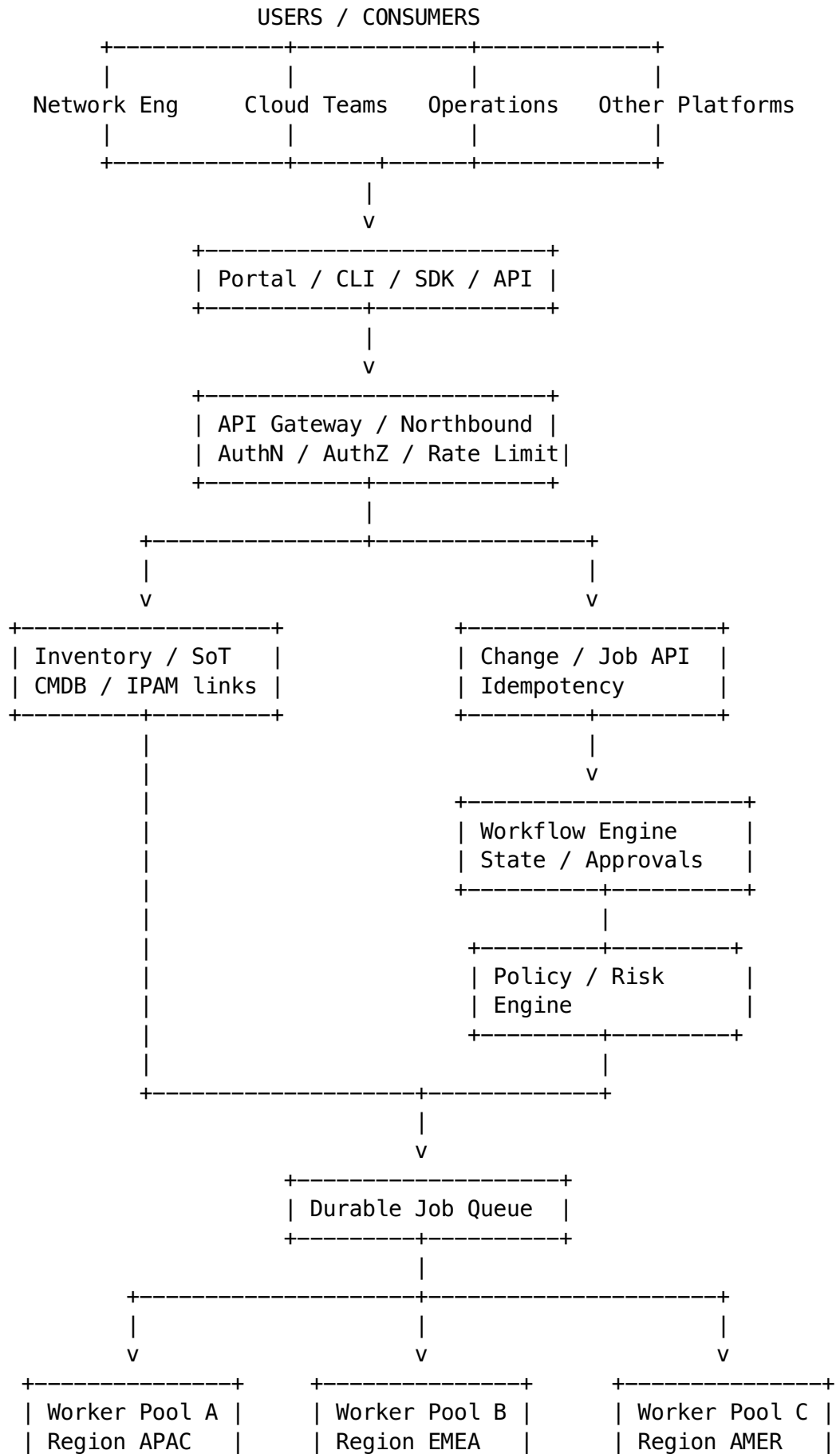
Step 5 — Architecture principles

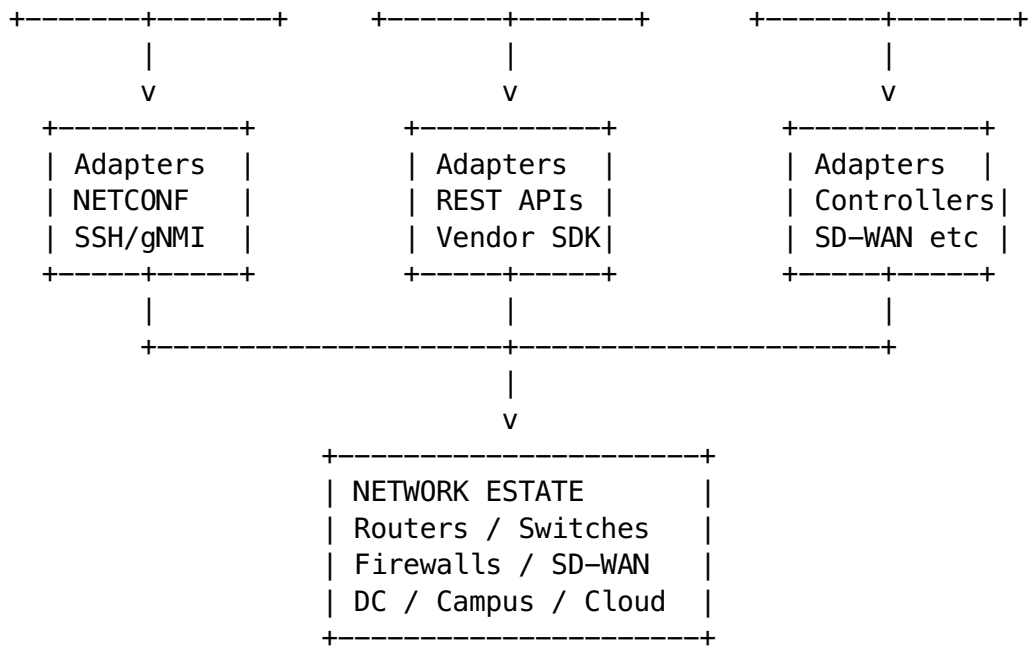
Choose principles:

API first
Source-of-truth driven
Asynchronous long-running work
Idempotent execution
Policy before execution
Small blast radius
Immutable audit trail
Observability by default
Region-aware design
Evolution over rewrite

Step 6 — Platform architecture

GLOBAL NETWORK AUTOMATION PLATFORM





CROSS-CUTTING PLATFORM CAPABILITIES

Secrets / PKI	Audit Store	Observability	Data Layer
Vault / HSM	Immutable log	Logs/Metrics	PostgreSQL
Rotation	Evidence	Traces/Alerts	Object Store
CI/CD IaC Security Scanning Test Automation Policy as Code			

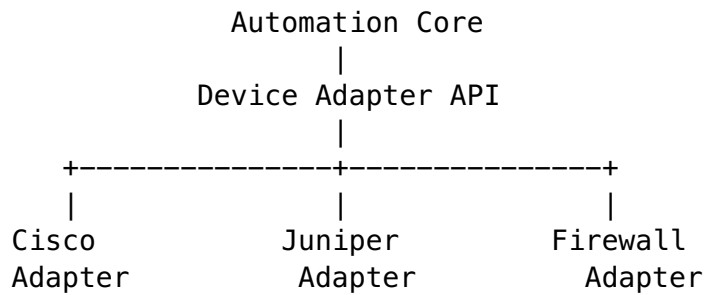
RESILIENCE / GOVERNANCE

Regional failure isolation
 +
 SLOs / error budgets
 +
 controlled deployment
 +
 change approvals
 +
 rollback / reconciliation
 +
 DR / backup / recovery

Step 7 — Network integration

Never assume all network devices behave identically.

Use adapter abstraction:



The core platform should not contain endless vendor-specific conditional logic.

Adapters translate platform intent into device capabilities.

Step 8 — API/platform design

Expose stable business-oriented operations.

Prefer:

```
POST /changes
GET  /changes/{id}
POST /changes/{id}/approve
POST /changes/{id}/cancel
GET  /devices/{id}
```

rather than exposing raw device implementation details.

Long-running changes return:

```
job_id
```

instead of holding HTTP connections open.

Step 9 — Kubernetes/cloud

Kubernetes provides:

- workload scheduling
- worker scaling
- rollout mechanisms
- service discovery
- deployment standardisation

But Kubernetes itself does not provide application-level resilience.

For example:

```
Pod restart ≠ workflow recovery
```

Durable job state must survive pod failure.

Step 10 — Data architecture

Different information requires different persistence.

Example:

PostgreSQL

- jobs
- approvals
- inventory metadata

Object storage

- large configuration snapshots
- reports

Telemetry system

- metrics
- high-volume device events

Audit store

- immutable security/change evidence
-

Do not put everything into one database merely because it is convenient initially.

Step 11 — Security architecture

Use:

Enterprise identity

↓

API authentication

↓

Role / policy authorization

↓

Workflow authorization

↓

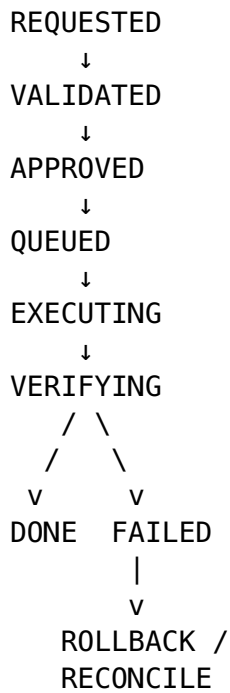
Device credentials

Key principles:

- no credentials in source code
 - central secret management
 - credential rotation
 - least privilege
 - short-lived credentials where possible
 - strong audit trail
-

Step 12 — Reliability model

An automation request should progress through an explicit state machine.



This makes failure behaviour understandable.

Step 13 — Controls

Risk-based policy can determine required controls.

Example:

Change: update lab switch

Risk: LOW

→ automated approval

Change: access switch

Risk: MEDIUM

→ standard workflow

Change: core network routing

Risk: HIGH

→ peer + change-manager approval

→ canary deployment

→ enhanced monitoring

Controls become part of the platform rather than external paperwork.

Step 14 — CI/CD

Code path:

Commit
↓
Unit tests
↓
Integration tests
↓
Security scans
↓
Policy checks
↓
Build artifact
↓
Staging
↓
Canary
↓
Production

Network automation logic also needs testing.

Possible environments:

mock device
virtual network lab
staging devices
canary production devices

Step 15 — Observability

Track technical metrics:

API latency
queue depth
worker saturation
job failures
retry count
device latency
controller errors

Track service outcomes:

successful automation %
change failure %
rollback %
time-to-complete

And business outcomes:

manual hours saved
provisioning lead time
automation adoption

Step 16 — Ownership

One possible ownership model:

Platform team

Owns:

- workflow engine
- APIs
- queue
- shared platform capabilities

Network domain teams

Own:

- device policy
- adapter/domain logic
- network validation

SRE

Owns jointly with platform:

- SLO implementation
- observability
- production readiness

Security

Owns:

- security standards
- control requirements

Risk

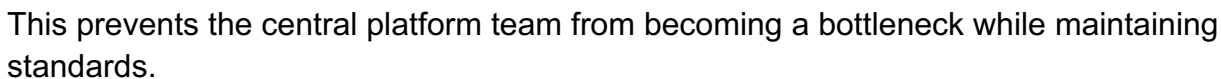
Owns:

- risk framework
- evidence requirements

But final operational accountability must still be explicit.

Step 17 — Engineering operating model

Prefer federated contribution with central platform ownership.



Measure cost by capability.

```
cost / managed device
cost / automation job
cost / telemetry GB
```

Platform engineering cost
+
Cloud/infrastructure cost
+
Vendor licences
+
Operations
-
Manual effort removed
-
Incident reduction
-
Faster business delivery

Not every platform must directly generate revenue to create economic value.

Scale different layers independently.

API traffic ↑
→ API pods ↑

Jobs ↑
→ workers ↑

Telemetry ↑
→ telemetry pipeline ↑

Inventory ↑
→ database capacity ↑

Avoid scaling the entire platform as one giant unit.

Step 20 — Migration

Do not migrate 100,000 devices simultaneously.

Suggested evolution:

Phase 1
Inventory + read-only discovery

Phase 2
Low-risk configuration

Phase 3
Standard changes

Phase 4
High-risk production automation

Phase 5
Closed-loop remediation

Use existing scripts during transition.

Gradually strangle them.

Step 21 — Long-term evolution

Over several years the platform may evolve from:

Scripts

↓

Central workflows

↓

API platform

↓

Declarative automation

↓

Policy-driven automation

↓

Closed-loop automation

But each step should be driven by actual capability requirements.

Do not jump directly to sophisticated intent-based automation while basic inventory accuracy remains poor.

Bringing the six days together

The previous topics now converge.

DAY 1

Networking fundamentals

|

v

DAY 2

Routing / DC / WAN architecture

|

v

DAY 3

Network automation platform

|

v

DAY 4

Cloud / Kubernetes / distributed systems

|

v

DAY 5

Reliability / security / resilience

|

v

DAY 6

Strategy / architecture / governance / leadership

Day 6 is therefore not another isolated technology topic.

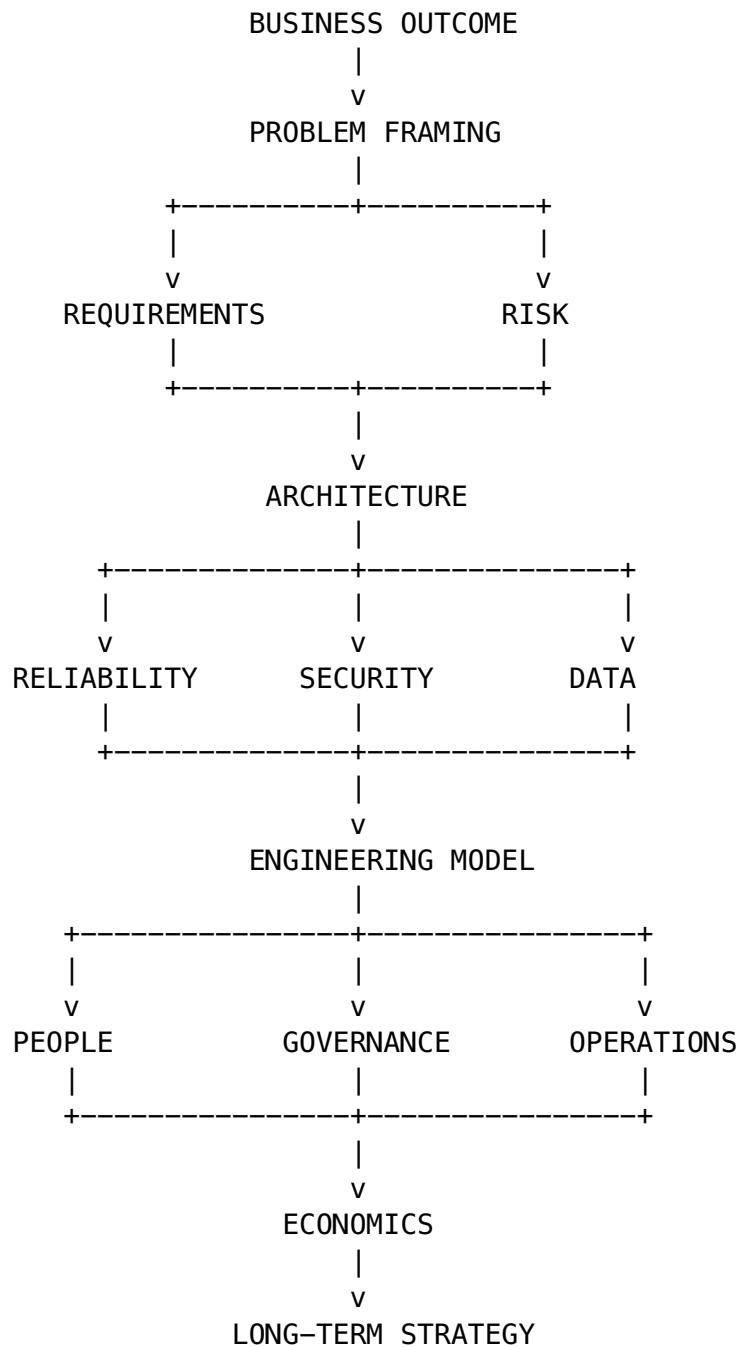
It asks:

| How do you combine everything underneath into a sustainable engineering capability?

The core Lead/VP mental model

A strong Lead/VP engineer does not think only about technology.

The complete picture is:



The important progression is:

Good engineer

= builds things correctly

Strong senior engineer

= designs systems correctly

Strong Staff/Principal engineer

= shapes systems that many teams can evolve

Strong Lead/VP engineer

= connects technology, architecture, reliability,
security, risk, economics, people and strategy
into one sustainable engineering system

The key question at this level is therefore rarely:

| **"What is the best technology?"**

It is much more often:

| **"Given our business objective, operating environment, people, constraints, risk tolerance and expected future, what is the simplest architecture and operating model that can deliver the outcome safely and evolve without trapping us?"**

That is the essence of technical leadership at platform and enterprise scale.